Prueba de endpoints – CQRS y Event Sourcing

Repositorio: `github.com/Juanpapb0401/cqrs-event-sourcing-kafka` (fork de jrquinte/cqrs-event-sourcing-kafka)

Herramienta: Postman. Command API en `localhost:5002` (el puerto 5000 lo ocupa macOS/AirPlay) y Query API en `localhost:5001`.

Infraestructura: Kafka, MongoDB (event store) y MySQL (read DB) con Docker Compose.

Las URLs de Postman usan variables de colección (`{{cmd}}`, `{{query}}`, `{{accountId}}`, `{{accountHolder}}`, `{{balanceFilter}}` …); en cada sección se muestra la URL ya resuelta.

Orden de pruebas: Open → Deposit → Withdraw → Lookups → Close → Lookup → Restore.

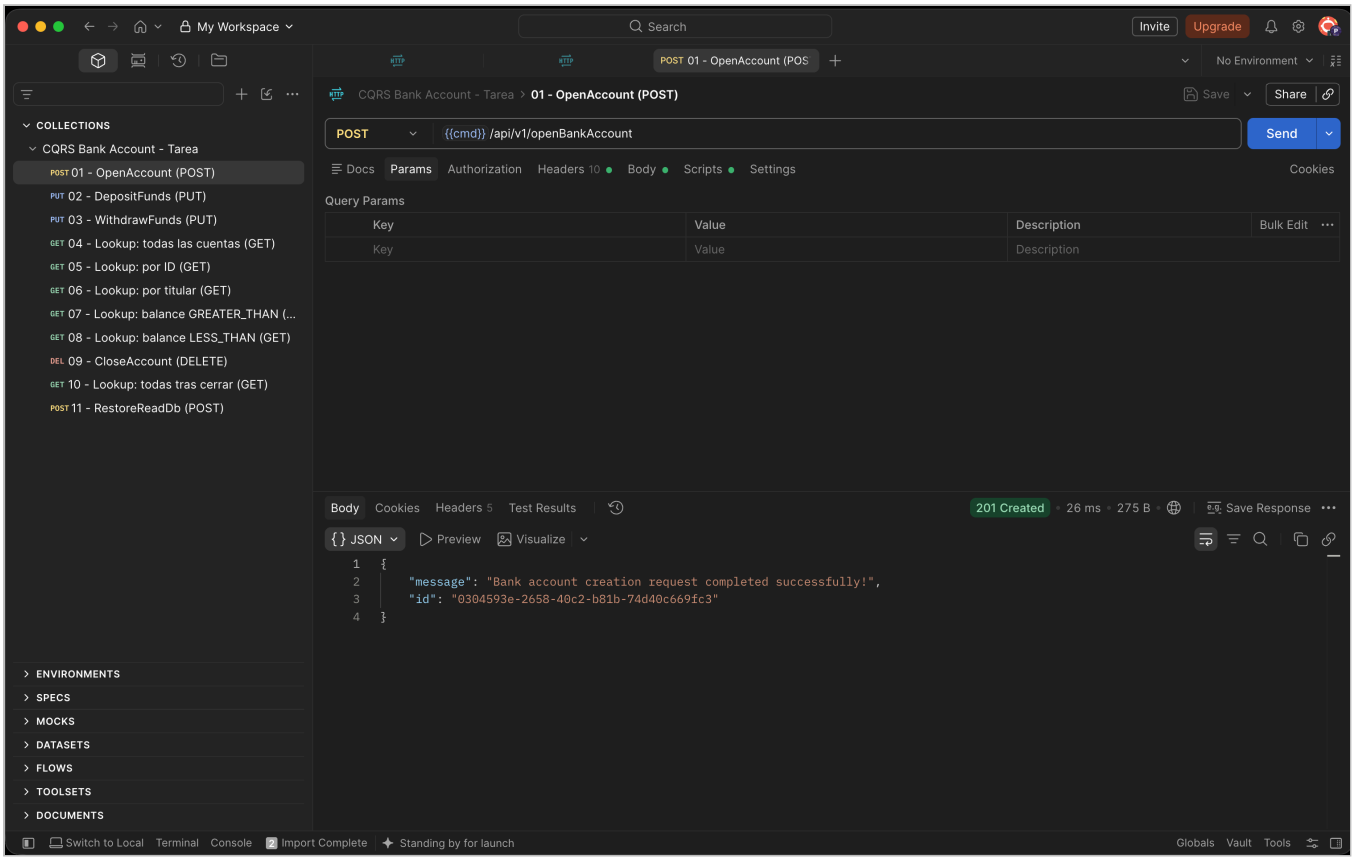
1. OpenAccountController

Método y URL: `POST http://localhost:5002/api/v1/openBankAccount`

Cuerpo:

```
{
  "accountHolder": "John Doe",
  "accountType": "SAVINGS",
  "openingBalance": 1000
}
```

Resultado: 201 Created. Crea la cuenta y devuelve el ID generado (UUID): 0304593e-2658-40c2-b81b-74d40c669fc3. Se guarda en la variable accountId para las demás pruebas.



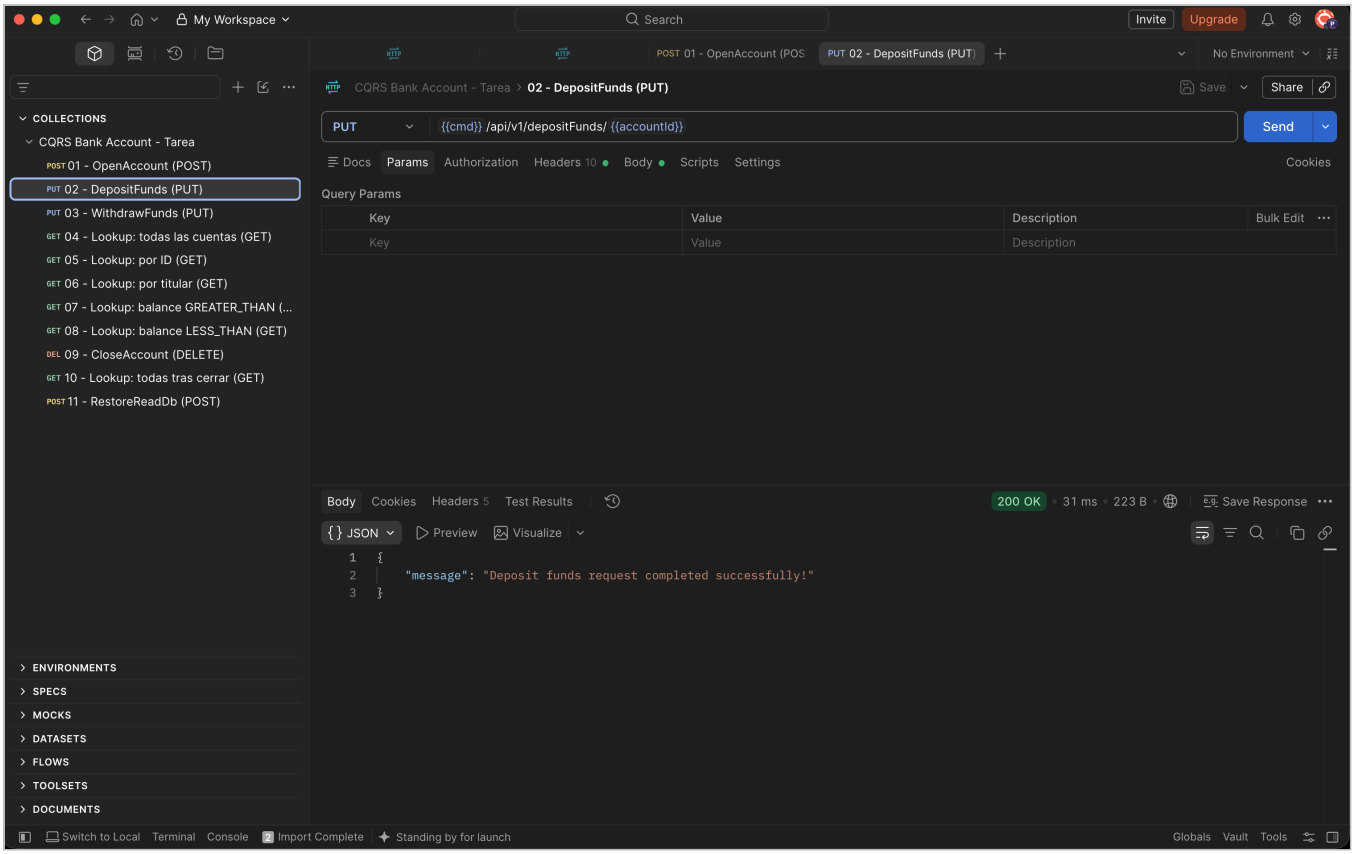
2. DepositFundsController

Método y URL: `PUT http://localhost:5002/api/v1/depositFunds/0304593e-2658-40c2-b81b-74d40c669fc3`

Cuerpo:

```
{
  "amount": 500
}
```

Resultado: 200 OK. Deposita 500 en la cuenta (balance 1000 → 1500).



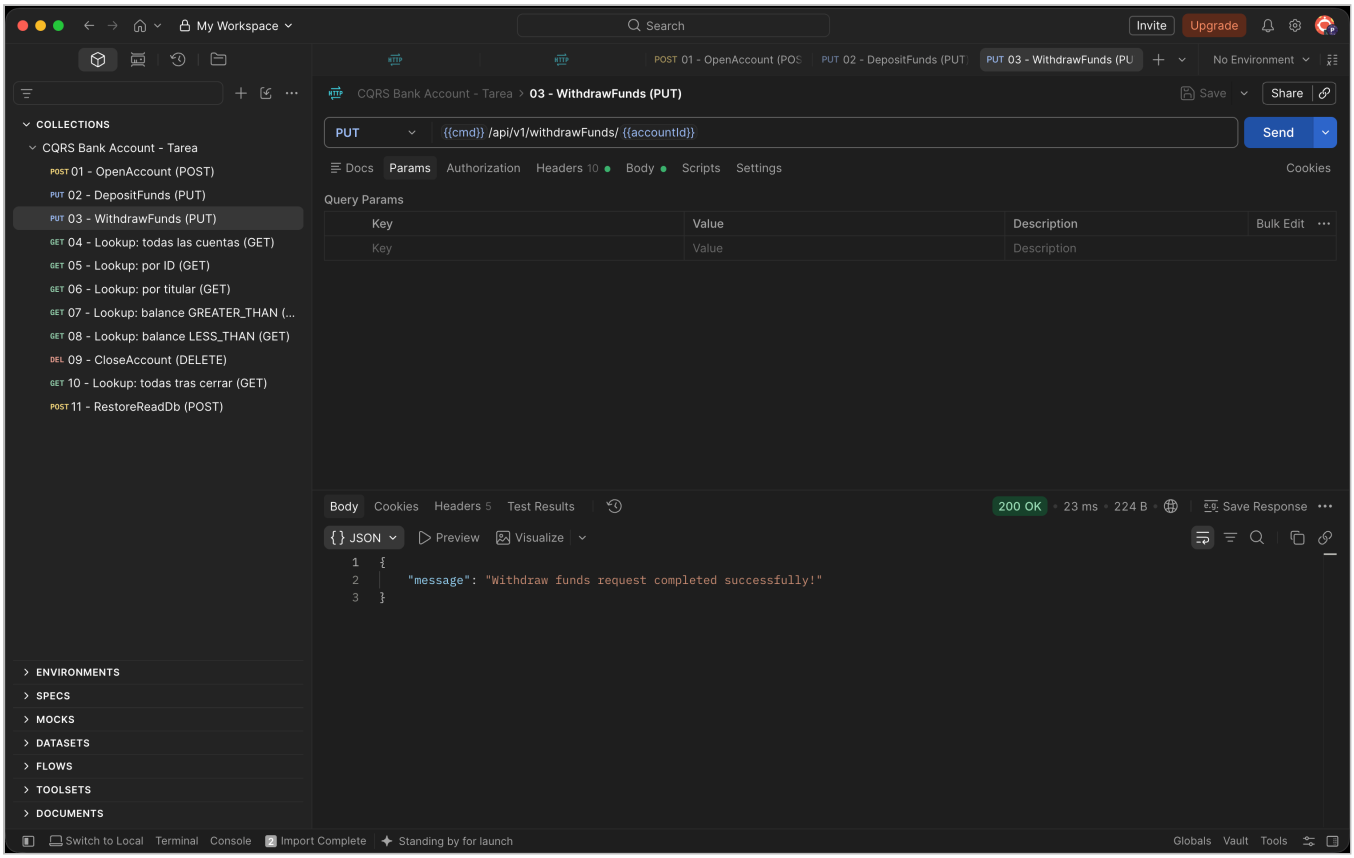
3. WithdrawFundsController

Método y URL: `PUT http://localhost:5002/api/v1/withdrawFunds/0304593e-2658-40c2-b81b-74d40c669fc3`

Cuerpo:

```
{
  "amount": 200
}
```

Resultado: 200 OK. Retira 200 de la cuenta (balance 1500 → 1300).



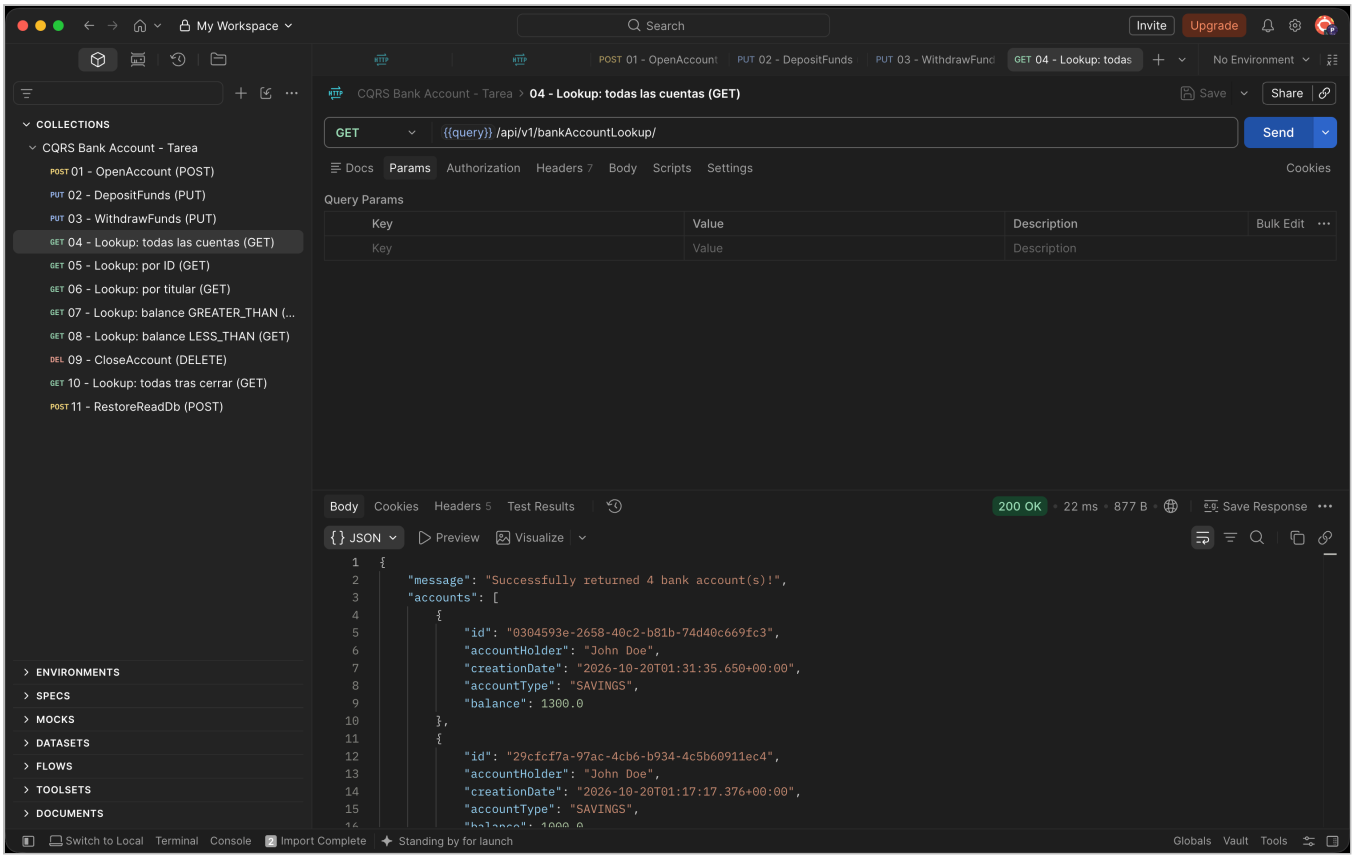
4. AccountLookupController – todas las cuentas

Método y URL: `GET http://localhost:5001/api/v1/bankAccountLookup/`

Cuerpo:

—

Resultado: 200 OK. Lista todas las cuentas del read DB (4 en ese momento, incluida la cuenta de prueba con balance 1300.0).



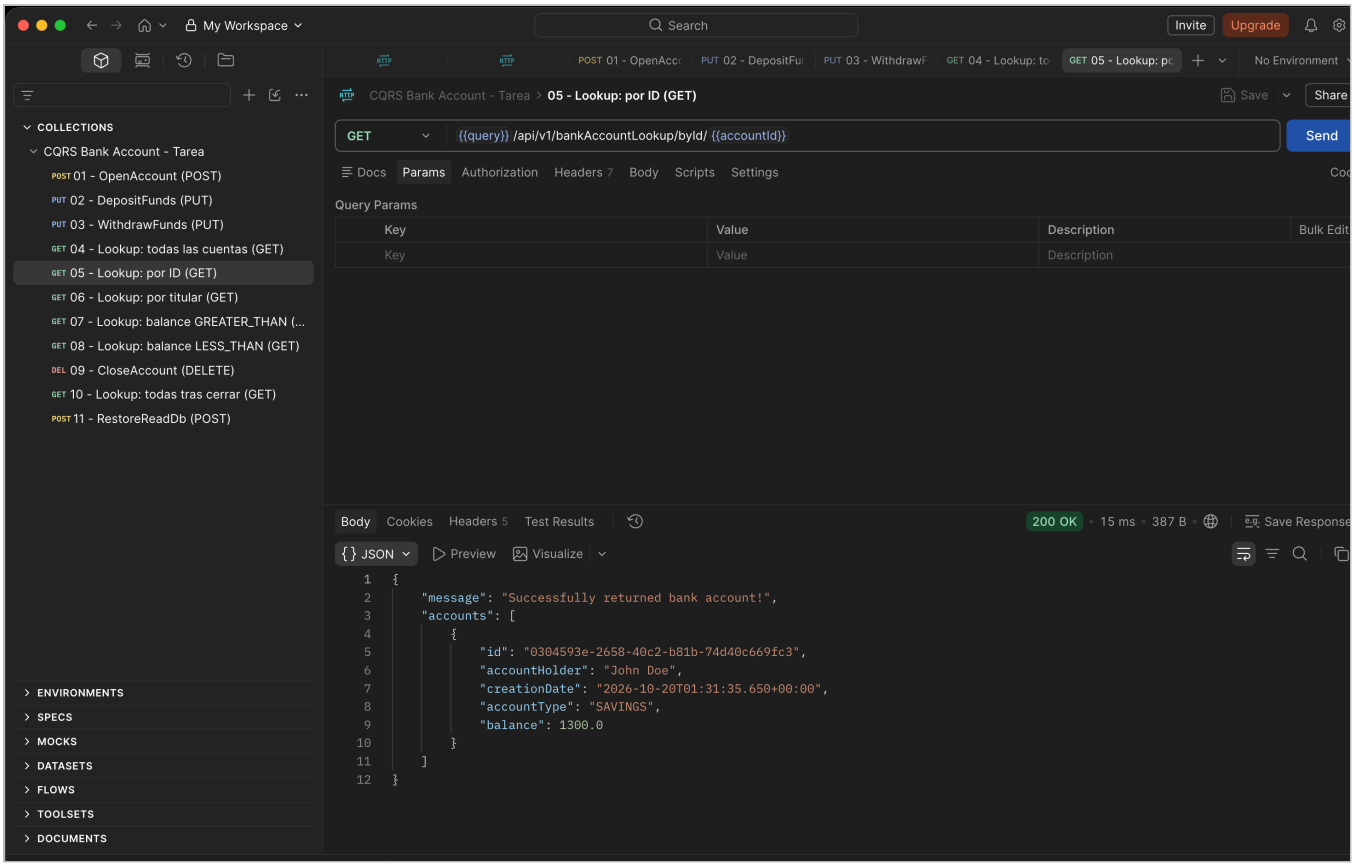
5. AccountLookupController – por ID

Método y URL: `GET http://localhost:5001/api/v1/bankAccountLookup/byId/0304593e-2658-40c2-b81b-74d40c669fc3`

Cuerpo:

```
–
```

Resultado: 200 OK. Devuelve la cuenta con balance 1300.0 (1000 + 500 – 200), lo que confirma que la proyección en la query side refleja los eventos.



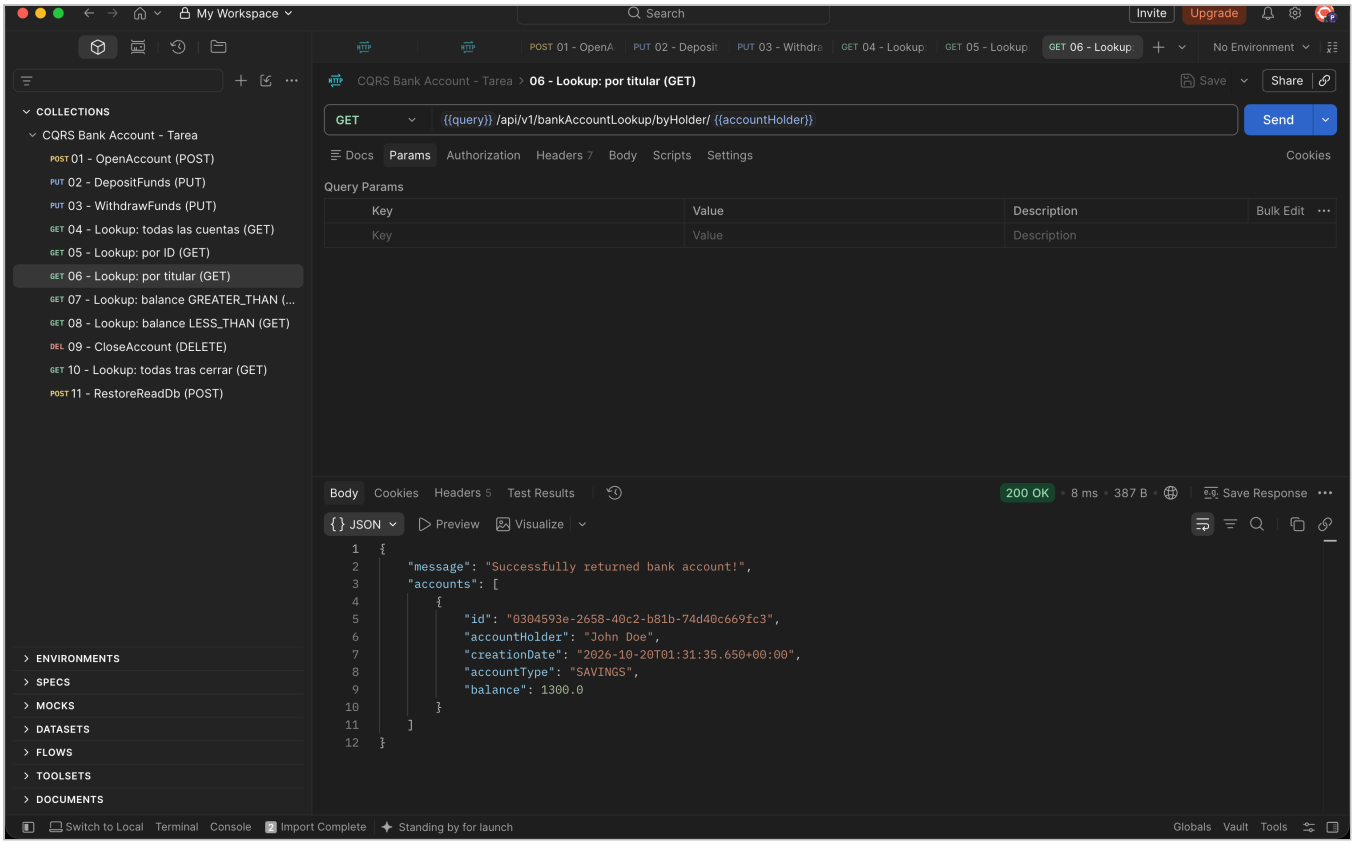
6. AccountLookupController – por titular

Método y URL: `GET http://localhost:5001/api/v1/bankAccountLookup/byHolder/John Doe`

Cuerpo:

```
–
```

Resultado: 200 OK. Devuelve la cuenta de John Doe. Nota: el repositorio usa Optional, por lo que con dos titulares iguales falla con NonUniqueResultException (error observado y corregido cerrando las cuentas duplicadas).



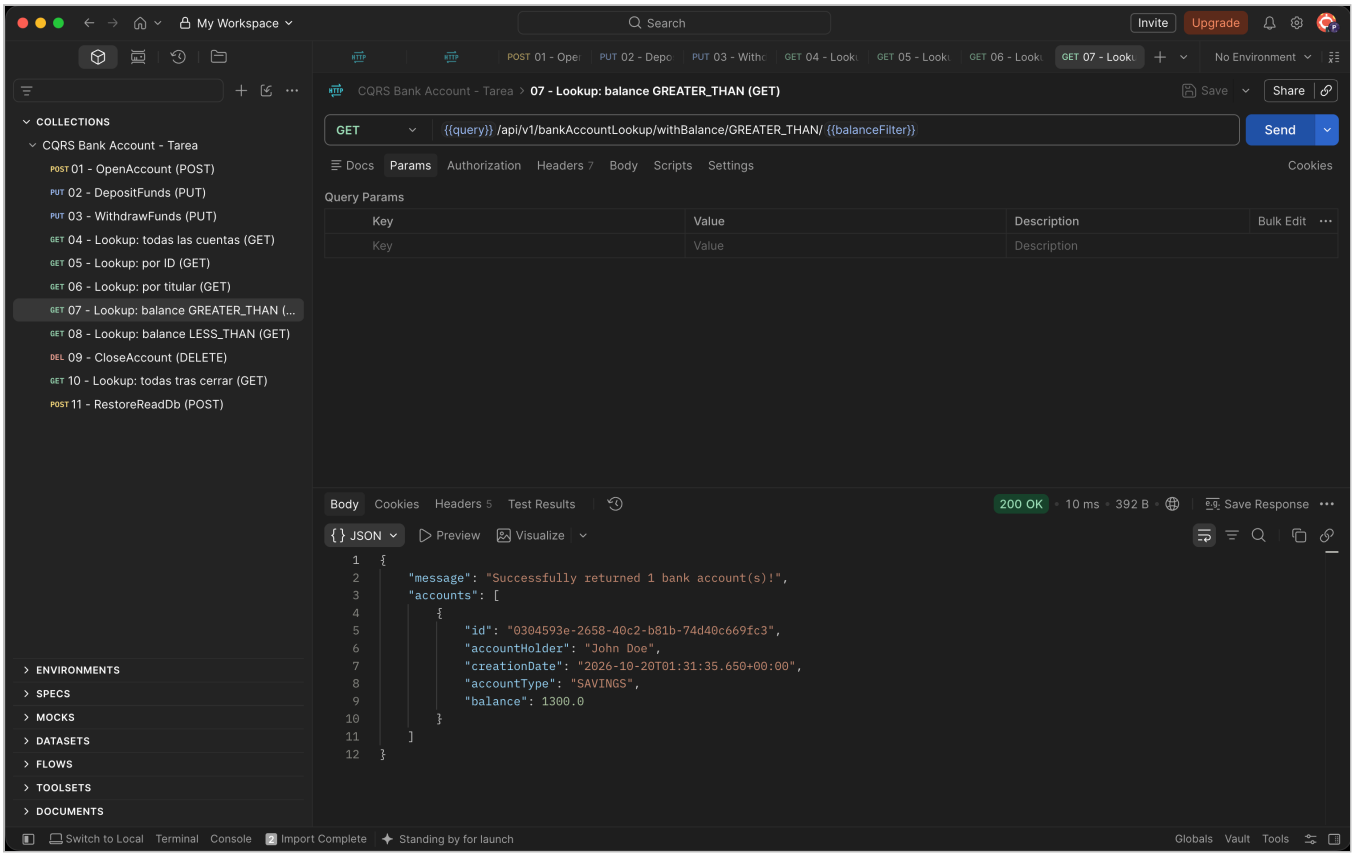
7. AccountLookupController – balance GREATER_THAN

Método y URL: `GET`
`http://localhost:5001/api/v1/bankAccountLookup/withBalance/GREATER_THAN/500`

Cuerpo:

—

Resultado: 200 OK. Devuelve 1 cuenta con balance mayor a 500 (John Doe, 1300.0).



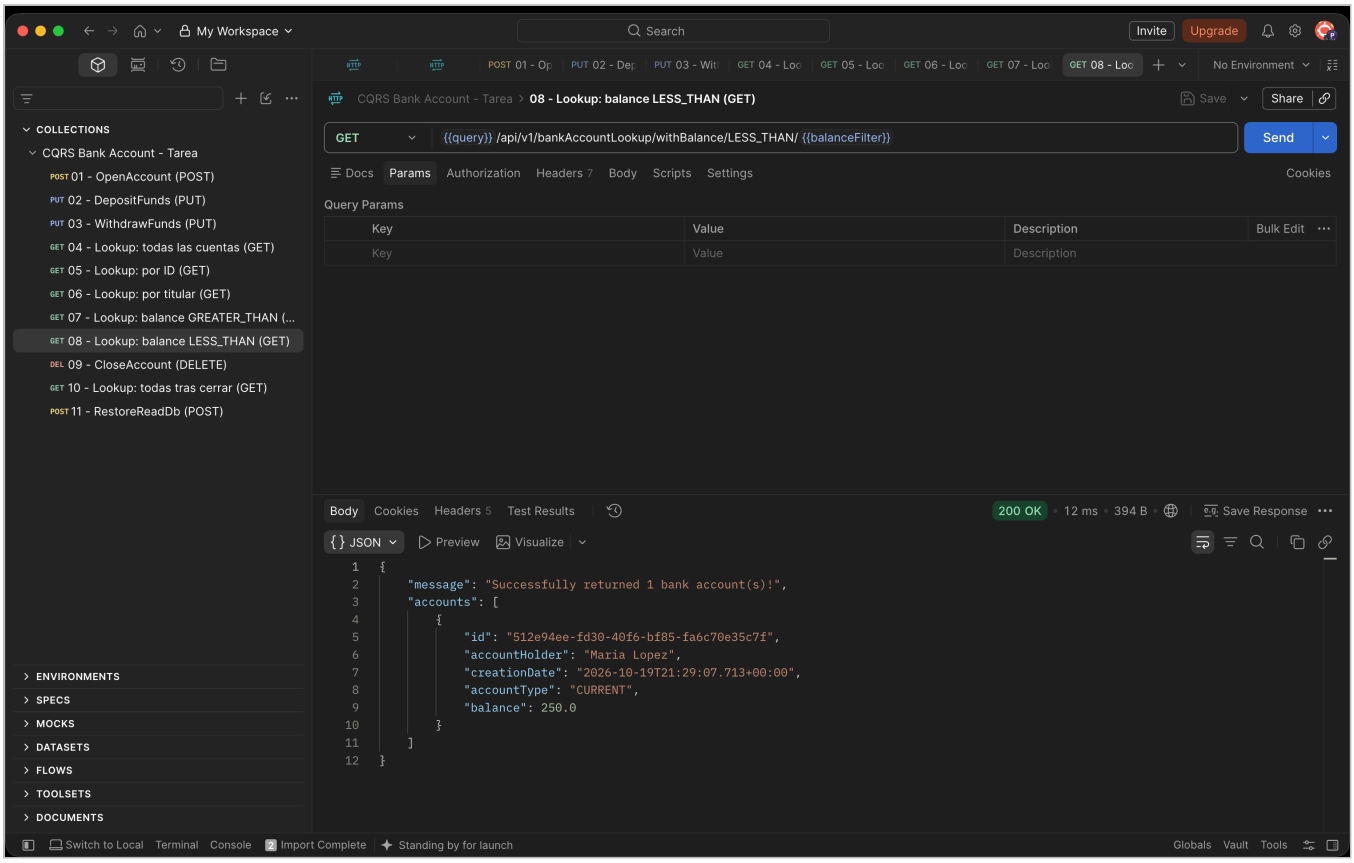
8. AccountLookupController – balance LESS_THAN

Método y URL: `GET http://localhost:5001/api/v1/bankAccountLookup/withBalance/LESS_THAN/500`

Cuerpo:

```
–
```

Resultado: 200 OK. Devuelve 1 cuenta con balance menor a 500 (Maria Lopez, 250.0).



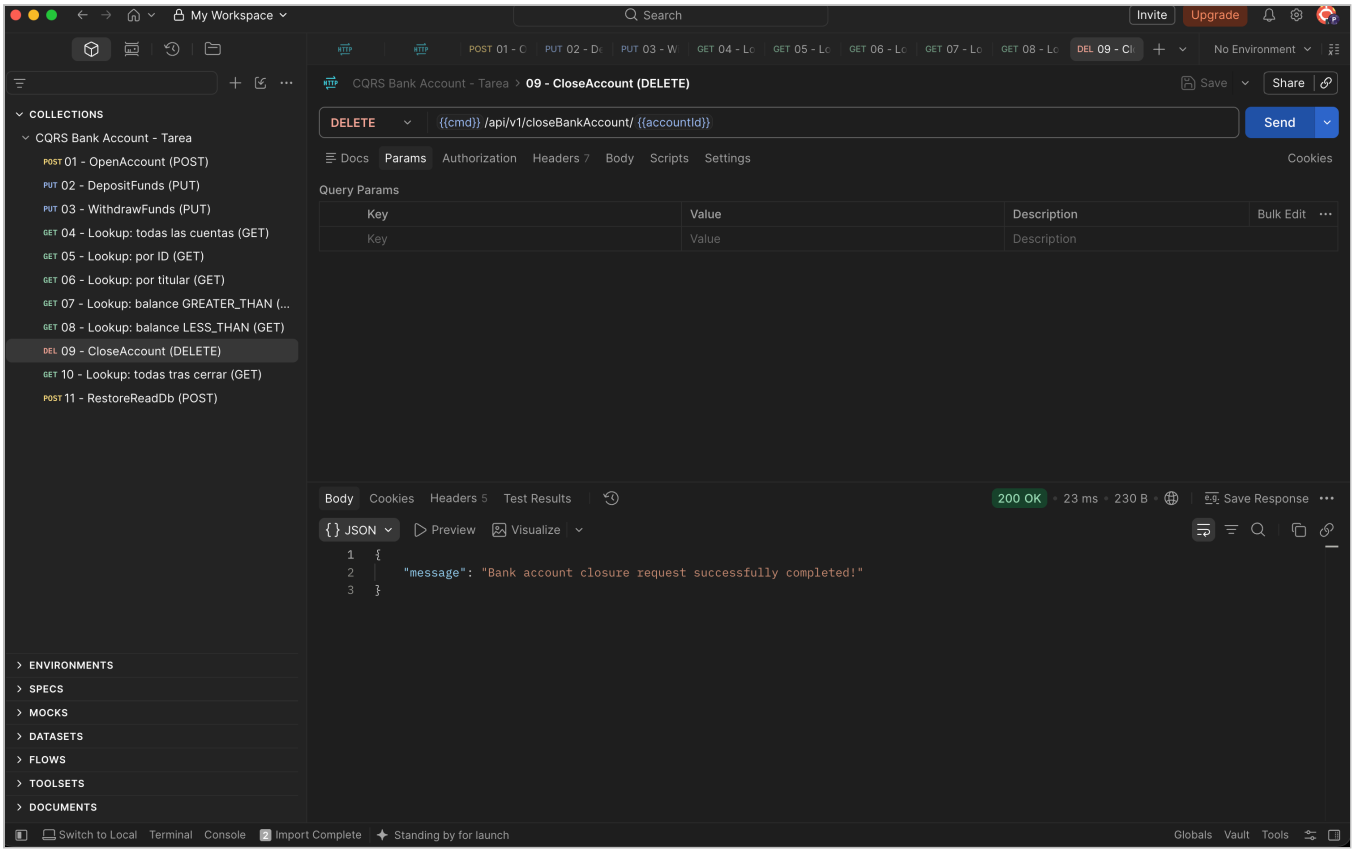
9. CloseAccountController

Método y URL: `DELETE http://localhost:5002/api/v1/closeBankAccount/0304593e-2658-40c2-b81b-74d40c669fc3`

Cuerpo:

```
–
```

Resultado: 200 OK. Cierra la cuenta; se emite AccountClosedEvent y la query side elimina la cuenta del read DB.



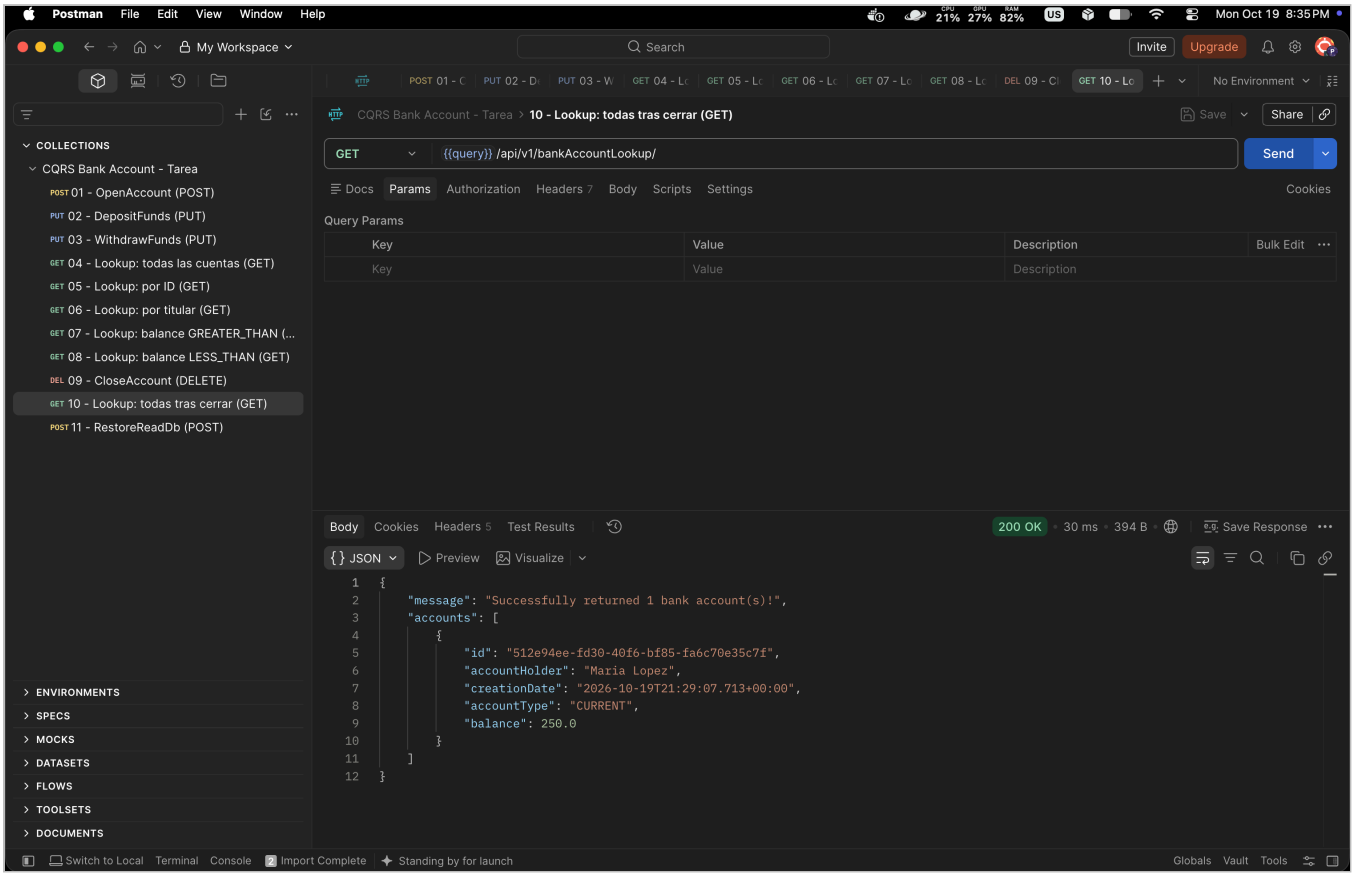
10. AccountLookupController – todas, tras cerrar

Método y URL: `GET http://localhost:5001/api/v1/bankAccountLookup/`

Cuerpo:

—

Resultado: 200 OK. Solo queda 1 cuenta (Maria Lopez): la cuenta cerrada ya no aparece.



11. RestoreReadDbController

Método y URL: `POST http://localhost:5002/api/v1/restoreReadDb`

Cuerpo:

```
–
```

Resultado: 201 Created. Republica los eventos del event store (MongoDB) al topic único para reconstruir el read DB.

